Day 4 - OOPs Encapsulation

ITSRUNTYM

1. Encapsulation
Definition:
Encapsulation is the process of **wrapping data (variables)** and **code (methods)** into a **single unit** (i.e., a class) and **restricting direct access** to some of the object's components. It is achieved using:

- **Access modifiers**
- **Getters and setters**

It protects internal object state from unintended or harmful changes.

Real-Life Example:
Think of a **Bank ATM Machine**:

- You interact with it via a screen & buttons (**public methods**)
- You don't know or access the internal circuits (**private data**)
- The bank hides your account data, only showing your balance (**encapsulation of data**)

Code Example (Java):

```
class BankAccount {
    // Private data members (data hiding)
    private String accountHolder;
    private double balance;

// Constructor
    public BankAccount(String accountHolder, double balance) {
        this.accountHolder = accountHolder;
        this.balance = balance;
    }
// Public getter
    public double getBalance() {
        return balance;
    }
// Public setter
    public void deposit(double amount) {
        if (amount > 0) {
```

```
            balance += amount;
        }
    }
    public void withdraw(double amount) {
        if (amount > 0 && balance >= amount) {
            balance -= amount;
        }
    }
}
```

Key Concepts:

Concept	Keyword	Role
Data Hiding	private	Prevents external access
Encapsulation	class, getters/setters	Controls access via methods

Benefits of Encapsulation:

- **Data security** (can't access directly)
- **Flexibility** (change internal implementation without affecting users)
- **Validation logic** (can add checks inside setters)
- **Code maintenance** (centralized control)

When to Use:

- Whenever you want **controlled access** to data
- When creating **reusable and maintainable components**
- Always keep variables private and provide public getters/setters if needed

2. Access Modifiers in Java (Detailed Explanation)

Access modifiers control **which classes or packages can access a class, variable, method, or constructor**. These are crucial for implementing **encapsulation**, **security**, and **code modularity**.

Java Provides 4 Access Levels:

Modifier	Accessible From	Typically Used For
private	**Within the same class only**	Sensitive data, internal logic

default	**Within the same package**	Package-level design, default access
protected	**Same package + child classes (even in different pkg)**	Inheritance, extension
public	**Accessible from anywhere**	Shared APIs, libraries

Let's Break Them Down with Examples

1. private – Most Restrictive

- The member is accessible **only within the class**.
- Used to **hide internal data** (i.e., encapsulation).
- Not accessible from other classes — not even subclasses.

```
public class Student {
   private String name; // Accessible only within Student class
private void printName() {
      System.out.println(name);
   }
}
class Test {
   public static void main(String[] args) {
      Student s = new Student();
      // s.name = "Karan";    Error: name is private
      // s.printName();       Error: private method
   }
}
```

Use private when:

- You don't want external classes to modify your data directly.
- You want to expose data only through getters/setters.

2. *Default* (Package-Private) – No Modifier

- No keyword = default access.
- Accessible **only within the same package**.
- Not accessible from **other packages**.

```
class Student {
   String course; // Default access (package-private)
```

```
}

// File in same package
class Test {
    public static void main(String[] args) {
        Student s = new Student();
        s.course = "Java"; //  Accessible within the package
    }
}

// File in another package
package another;
import mypackage.Student;

class OtherTest {
    public static void main(String[] args) {
        Student s = new Student();
        // s.course = "Python";  Error: not accessible from another package
    }
}
```

Use *default* when:

- You want to share members only with other classes in the **same package** (like a module).

3. protected – Package + Subclass Access

- Accessible within:
  - The **same package**
  - Any **subclass** (even in a different package)

```
class Student {
    protected int age; // Accessible in subclass or same package
}

// Same package
class Test {
    public static void main(String[] args) {
        Student s = new Student();
        s.age = 20; //  Accessible within package
    }
}
```

```
// Different package with inheritance
package another;
import mypackage.Student;

class SubStudent extends Student {
    void display() {
        System.out.println(age); //  Allowed in subclass
    }
}
```

Use protected when:

- You want subclasses (in or outside the package) to access members.
- You're following **inheritance** and **OOP reusability**.

4. **public – Least Restrictive**
   - Accessible from **anywhere** (same or different package).
   - Used for APIs, library classes, methods meant for open usage.

```
public class Student {
    public int id; // Accessible everywhere
}


class Test {
    public static void main(String[] args) {
        Student s = new Student();
        s.id = 101; //  No restriction
    }
}
```

Use public when:

- You are exposing **APIs**, **utility methods**, or **entry point classes** (public static void main()).
- You want the class or method to be reusable from any package.

Combined Example:

```
public class Student {
    private String name;        // Only within class
    protected int age;          // Package + subclass
```

```
    String course;              // Package-private (default)
    public int id;              // Accessible everywhere

    public void displayInfo() {
        System.out.println("ID: " + id);
        System.out.println("Course: " + course);
        System.out.println("Age: " + age);
        System.out.println("Name: " + name);
    }
}


// Access from another class in same package
class Test {
    public static void main(String[] args) {
        Student s = new Student();
        s.id = 1;           //  public
        s.course = "Java";  //  default
        s.age = 21;         //  protected (same package)

        // s.name = "Karan"; //  private, not accessible
    }
}
```

How Access Modifiers Help in:

Concept	Access Modifier	Purpose
Encapsulation	private	Hide internal state
Controlled access	protected / public	Allow safe interaction
Modularization	default	Share only within modules
Reusability	public	Share across app/ libraries

Best Practices:

- Always **keep data members private**.
- Use **getters/setters** for access.
- Mark helper methods as **private or protected**.
- Expose only **what's needed** to avoid tight coupling.

3. Getters and Setters

What are they?
Methods that allow **controlled access** to private fields.

Why needed?
Direct access is **dangerous**:

```
student.name = "";  // No validation!
```

With Getters/Setters:

```
public class Student {
    private String name;
// Getter
    public String getName() {
        return name;
    }
// Setter with validation
    public void setName(String name) {
        if(name != null && !name.isEmpty()) {
            this.name = name;
        }
    }
}
```

4. Abstraction

Definition:
Abstraction means hiding the **complex implementation details** and showing only the **essential features**.
"Focus on **what** an object does, not **how** it does it."

Real-Life Example:
- You **drive a car** (you know **how to drive**, not **how engine works**)
- You **click a button** on an app (you don't know the backend logic)

Achieved using:
1. **Abstract Classes**
2. **Interfaces**

4.1 Abstract Class
- Partial abstraction (can have method implementations)

- Use abstract keyword
- Can have constructors, instance variables, static methods

Example:

```
abstract class Animal {
    String name;
public Animal(String name) {
        this.name = name;
    }
// Abstract method
    public abstract void makeSound();
// Non-abstract method
    public void sleep() {
        System.out.println(name + " is sleeping...");
    }
}
class Dog extends Animal {
    public Dog(String name) {
        super(name);
    }
public void makeSound() {
        System.out.println(name + " barks");
    }
}
```

4.2 Interface

- Full abstraction (only method signatures before Java 8)
- No implementation allowed (until Java 8 default methods)
- Used to define **contracts** or capabilities

Example:

```
interface Flyable {
    void fly();  // Abstract method
}

class Bird implements Flyable {
    public void fly() {
        System.out.println("Bird is flying...");
    }
}
```

Interface Features:

Feature	Abstract Class	Interface
Multiple Inheritance	No	Yes (Java supports multiple interfaces)
Constructors	Yes	Yes
Default Methods	Yes	Yes (since Java 8)
Fields	Can have variables	public static final only

Benefits of Abstraction:

- Reduces complexity for user
- Helps in implementing **polymorphism**
- Promotes **loose coupling**
- Enhances **code reusability**

When to Use:

Scenario	Use
Partial abstraction	Abstract class
Full abstraction / multiple inheritance	Interface
You want default method body	Abstract class or interface (Java 8+)

What are Default Methods in Java?

A **default method** is a method in an **interface** that includes a **default implementation**.
Before Java 8, **interfaces could only declare methods (without a body)**.
From Java 8 onward, you can define **default methods** (with a body) inside interfaces.

Syntax:

```
interface InterfaceName {
    default void methodName() {
        // implementation
    }
}
```

Why Were Default Methods Introduced?

Problem:

- Prior to Java 8, if you added a new method to an interface, **all**

implementing classes broke because they had to implement the new method.

Solution:

- Default methods allow **backward compatibility**.
- You can add new methods to an interface **without affecting** existing implementations.

Real Example Before Java 8:

```
interface Vehicle {
    void start();
}

// Later you want to add `stop()` → breaks all classes!
interface Vehicle {       //  Problem!
    void start();
    void stop(); // All old classes must now implement this!
}
```

Java 8 Solution (Default Method):

```
interface Vehicle {
    void start();
default void stop() {
        System.out.println("Vehicle stopped.");
    }
}
```

Now, classes can **choose to override** the stop() method or use the **default** one.

Code Example:

```
interface Vehicle {
    void start();
default void stop() {
        System.out.println("Vehicle stopped");
    }
}
class Car implements Vehicle {
    public void start() {
        System.out.println("Car started");
```

```
    }
}
class Bike implements Vehicle {
    public void start() {
        System.out.println("Bike started");
    }
// Optional: override stop()
    public void stop() {
        System.out.println("Bike stopped using disc brake");
    }
}
public class Main {
    public static void main(String[] args) {
        Vehicle car = new Car();
        car.start(); // Car started
        car.stop();  // Vehicle stopped (default)
Vehicle bike = new Bike();
        bike.start(); // Bike started
        bike.stop();  // Bike stopped using disc brake (overridden)
    }
}
```

Relationship between Abstraction and Encapsulation

Aspect	Encapsulation	Abstraction
Focus	Internal data hiding	Hiding implementation logic
Achieved via	Access Modifiers, Getters/Setters	Abstract Classes, Interfaces
Goal	Protect data	Reduce complexity
Visibility	Data is hidden	Implementation is hidden

```
// Abstraction via Interface
interface BankAccount {
    void deposit(double amount);
    void withdraw(double amount);
    double getBalance();
}

// Encapsulation in Implementation
```

```
class SavingAccount implements BankAccount {
    private double balance;  //  Encapsulated

    @Override
    public void deposit(double amount) {
        if (amount > 0) balance += amount;
    }

    @Override
    public void withdraw(double amount) {
        if (amount <= balance) balance -= amount;
    }

    @Override
    public double getBalance() {
        return balance;
    }
}
```

Keywords Summary

Keyword	Purpose
private	Encapsulation – data hiding
public	Access control
protected	Inheritance-friendly access
abstract	Abstraction through class/method
interface	Full abstraction, contract
getter/setter	Controlled access to data
final	Prevent inheritance (used to secure code)

Real-World Scenarios

Scenario	Concept Applied
Banking System	Encapsulation (secure data), Abstraction (transactions)
Payment Gateways	Interface (common processPayment() method)
Web Frameworks	Abstract classes for controllers

Mobile Apps	Abstract base class for screens or adapters

Design Principles & Best Practices

- Always use **private fields** and expose data via **getters/setters**
- Use **interfaces** to define roles (Flyable, Serializable)
- Use **abstract classes** for common behavior (BaseController)
- Abide by **OOP principles**:
  - **SOLID**, especially:
    - **O**pen/Closed Principle
    - **L**iskov Substitution Principle
    - **D**ependency Inversion via interfaces

Summary Table:

Concept	Purpose	How	Real Life
Encapsulation	Protect data	private + getters/setters	ATM data access
Abstraction	Hide complexity	abstract class / interface	Car engine, remote
Getters/Setters	Controlled access	via methods	Set volume on TV
Access Modifiers	Control scope	public/private/ etc.	Room access levels
Abstract Class	Partial abstraction	abstract keyword	Shape base class
Interface	Full abstraction	interface	Flyable, Drawable